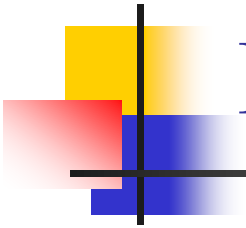
COURSE INFORMATION SHEET

Course Code: CS24201
Course Title: Data Structure and Algorithms
Pre-requisite(s): CS24101 Programming for Problem Solving Co- requisite(s): MA24205 Discrete Mathematics
Credits: L: 3 T: 0 P: 0
Class schedule per week:
Class:
Semester / Level: III Branch: CSE

MODULE	(NO. OF LECTURE HOURS)
Module – I **Basics of Data Structures and Algorithm, Time and Space Complexity, Asymptotic Notations, Time complexity analysis of non-recursive and recursive algorithms, Examples of multi-dimensional array, polynomial operations and sparse matrix**	**5**
Module – II **Singly Linked List: concept, representation and operations, Circular Linked List, Doubly Linked List, Multi-ply linked list and their Applications**	**8**
Module – III	**10**
Stack: basic operations using Array and LL, Queue: basic operations using Array and LL, Circular Queue and Variants of Queue, Stack and Queue Applications	
Module – IV **Binary Search, Hash Search, Selection Sort, Insertion Sort, Quick Sort, Merge Sort, Radix Sort, External Sorting: k-way merging approach, Analysis of Search and Sorting Algorithms**	**7**
Module – V **Basic concepts and terminologies of Binary Search Tree, Height Balanced Trees and Heap, Disjoint Set, Graph: concept and terminologies, Concept of Breadth First Search, Depth First Search, Spanning Tree.**	**10**

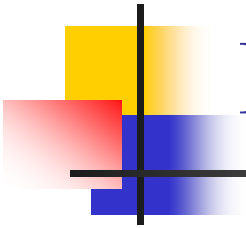
Introduction to Data Structures





Definition

- Data structure is representation of the logical relationship existing between individual elements of data.
- In other words, a data structure is a way of organizing al data items that considers not only the elements stored but also their relationship to each other.

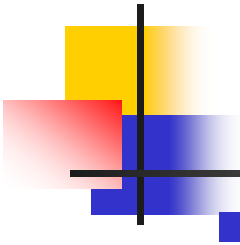


Introduction

- Data structure afects the design of both structural & functional aspects of a program.

  Program=algorithm + Data Structure

- You know that a algorithm is a step by step procedure to solve a particular function.



Introduction

- That means, algorithm is a set of instruction writen to carry out certain tasks & the data structure is the way of organizing the data with their logical relationship retained.
- To develop a program of an algorithm, we should select an appropriate data structure for that algorithm.
- Therefore algorithm and its associated data structures from a program.

Data Structure is a systematic way to organize data in order to use it eficiently.. Folowing terms are the foundation terms of a data structure..

Interface − Each data structure has an interface.. Interface represents the set of operations that a data structure supports.. An interface only provides the list of supported operations,, type of parameters they can accept and return type of these operations..

Implementation − implementation provides the internal representation of a data structure.. implementation also provides the definition of the algorithms used in the operations of the data structure..

Characteristics of a Data Structure

Correctness − Data structure implementation should implement its interface correctly..

Time Complexity − Running time or the execution time of operations of data structure must be as smal as possible..

Space Complexity − Memory usage of a data structure operation should be as litle as possible..

Need for Data Structure

- As applications are geting complex and data rich,, there are three common problems that applications face now-a-days..

Data Search − Consider an inventory of 1 milion(10^6) items of a store.. if the application is to search an item,, it has to search an item in 1 milion(10^6) items every time slowing down the search.. As data grows,, search wil become slower..

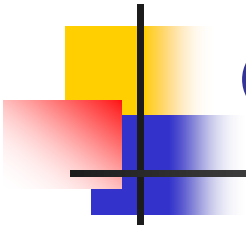
- **Processor speed** − Processor speed although being very high,, fals limited if the data grows to bilion records..

- **Multiple requests** − As thousands of users can search data simultaneously on a web server,, even the fast server fails while searching the data..

- To solve the above-mentioned problems,, data structures come to rescue.. Data can be organized in a data structure in such a way that al items may not be required to be searched,, and the required data can be searched almost instantly..

Basic Terminology

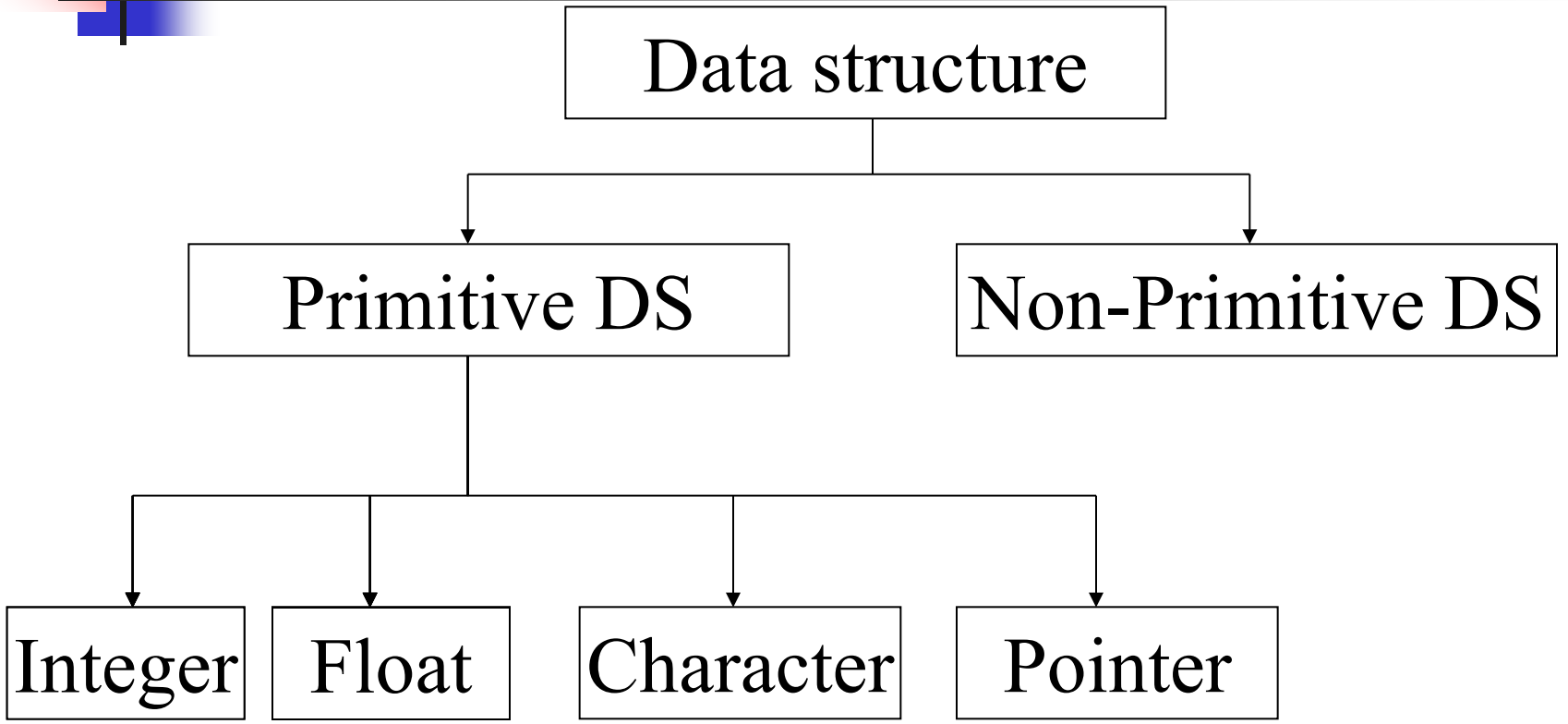
- **Data** − Data are values or set of values..
- **Data item** − Data item refers to single unit of values..
- **Group items** − Data items that are divided into sub items are called as Group items..
- **Elementary items** − Data items that cannot be divided are called as Elementary items..
- **Atribute and Entity** − An entity is that which contains certain attributes or properties,, which may be assigned values..
- **Entity Set** − Entities of similar attributes form an entity set..
- **Field** − Field is a single elementary unit of information representing an attribute of an entity..
- **Record** − Record is a collection of field values of a given entity..
- **File** − File is a collection of records of the entities in a given entity set..



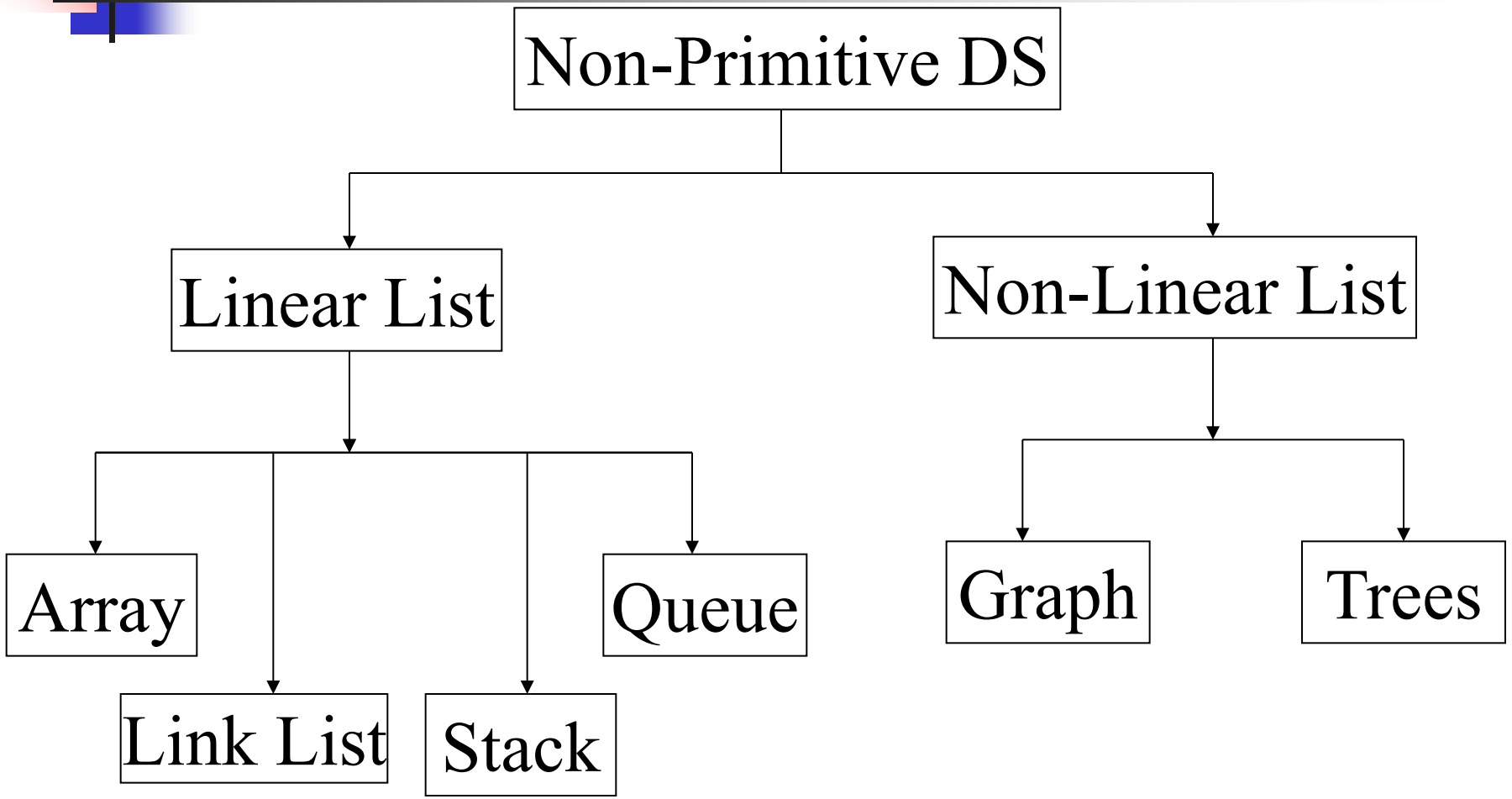
Classification of Data Structure

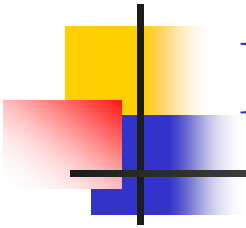
- Data structure are normaly divided into two broad categories:
  - Primitive Data Structure
  - Non-Primitive Data Structure

Classification of Data Structure



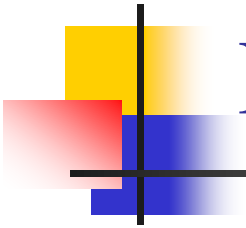
Classification of Data Structure





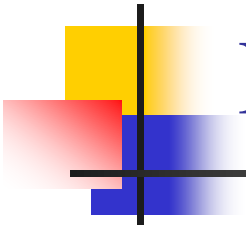
Primitive Data Structure

- There are basic structures and directly operated upon by the machine instructions.
- In general, there are diferent representation on diferent computers.
- Integer, Floating-point number, Character constants, string constants, pointers etc, fal in this category.



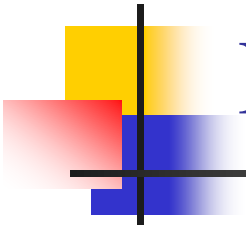
Non-Primitive Data Structure

- There are more sophisticated data structures.
- These are derived from the primitive data structures.
- The non-primitive data structures emphasize on structuring of a group of homogeneous (same type) or heterogeneous (diferent type) data items.



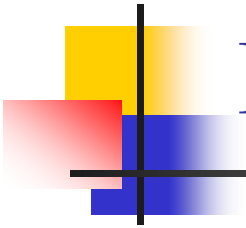
Non-Primitive Data Structure

- Lists, Stack, Queue, Tree, Graph are example of non-primitive data structures.
- The design of an eficient data structure must take operations to be performed on the data structure.



Non-Primitive Data Structure

- The most commonly used operation on data structure are broadly categorized into folowing types:
  - Create
  - Selection
  - Updating
  - Searching
  - Sorting
  - Merging
  - Destroy or Delete

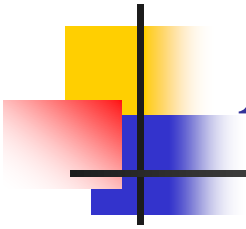


Diferent between them

- A primitive data structure is generaly a basic structure that is usualy built into the language, such as an integer, a float.
- A non-primitive data structure is built out of primitive data structures linked together in meaningful ways, such as a or a linked-list, binary search tree, AVL Tree, graph etc.

Description of various Data Structures : Arrays

- An array is defined as a set of finite number of homogeneous elements or same data items.
- It means an array can contain one type of data only, either al integer, al float-point number or al character.

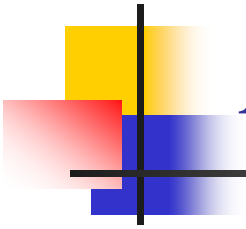


Arrays

- Simply, declaration of array is as folows:

  int arr[10]

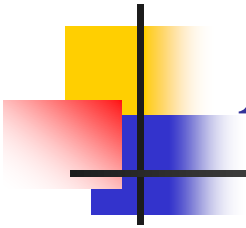
- Where int specifies the data type or type of elements arrays stores.
- “arr” is the name of array & the number specified inside the square brackets is the number of elements an array can store, this is also caled sized or length of array.



Arrays

- Folowing are some of the concepts to be remembered about arrays:
  - The individual element of an array can be accessed by specifying name of the array, folowing by index or subscript inside square brackets.
  - The first element of the array has index zero[0]. It means the first element and last element wil be specified as:arr[0] & arr[9]

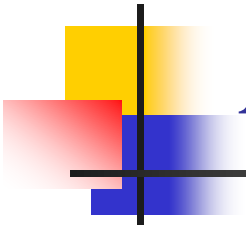
    Respectively.



Arrays

- The elements of array wil always be stored in the consecutive (continues) memory location.
- The number of elements that can be stored in an array, that is the size of array or its length is given by the folowing equation:

  (Upperbound-lowerbound)+1

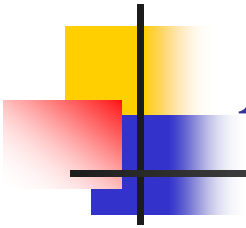


Arrays

- For the above array it would be

  (9-0)+1=10,where 0 is the lower bound of array and 9 is the upper bound of array.

- Array can always be read or writen through loop. If we read a one-dimensional array it require one loop for reading and other for writing the array.



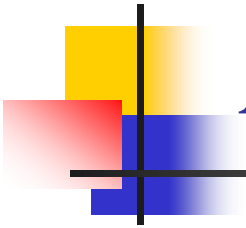
Arrays

- For example: Reading an array

```
For(i=0;i<=9;i++)
    scanf("%d",&arr[i]);
```

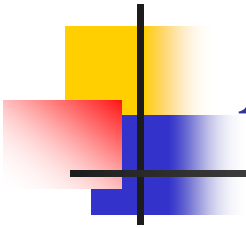
- For example: Writing an array

```
For(i=0;i<=9;i++)
    printf("%d",arr[i]);
```



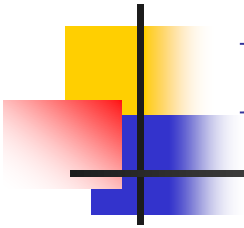
Arrays

- If we are reading or writing two-dimensional array it would require two loops. And similarly the array of a N dimension would required N loops.
- Some common operation performed on array are:
  - Creation of an array
  - Traversing an array



Arrays

- Insertion of new element
- Deletion of required element
- Modification of an element
- Merging of arrays



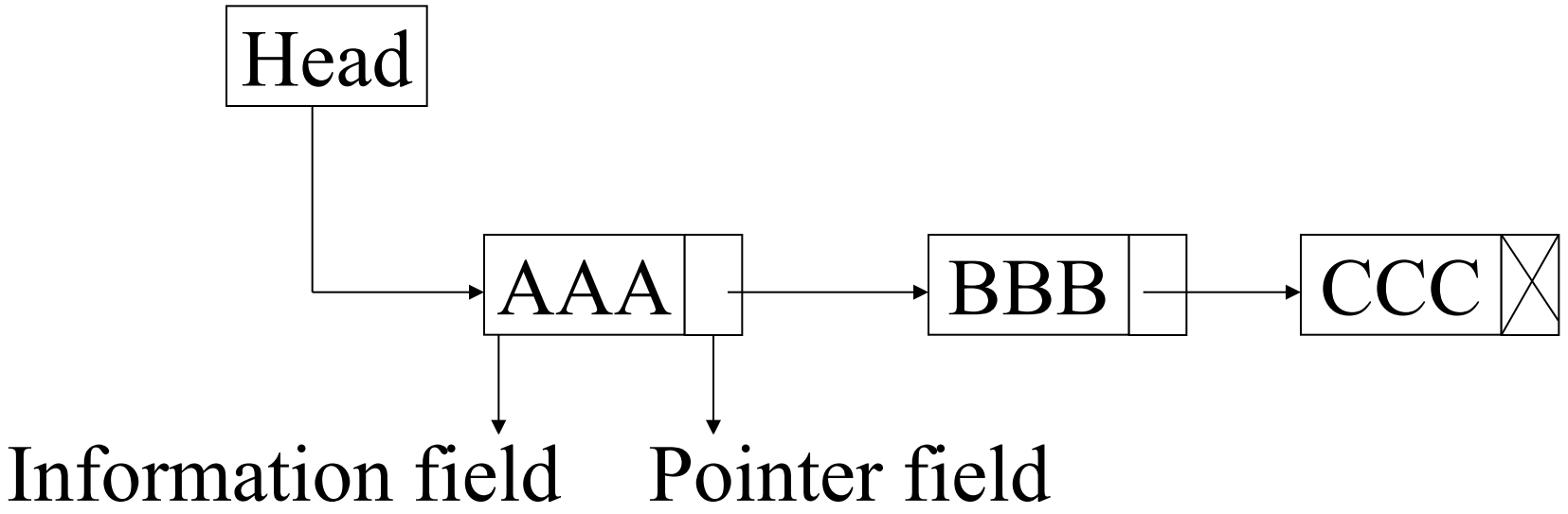
Lists

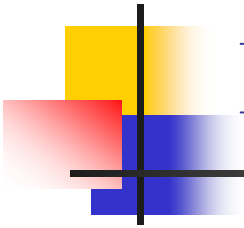
- A lists (Linear linked list) can be defined as a colection of variable number of data items.
- Lists are the most commonly used non-primitive data structures.
- An element of list must contain at least two fields, one for storing data or information and other for storing address of next element.
- As you know for storing address we have a special data structure of list the address must be pointer type.

Lists

- Technicaly each such element is referred to as a node, therefore a list can be defined as a colection of nodes as show below:

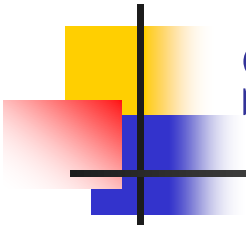
[Linear Liked List]





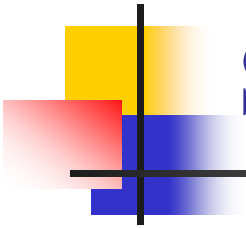
Lists

- Types of linked lists:
  - Single linked list
  - Doubly linked list
  - Single circular linked list
  - Doubly circular linked list



Stack

- A stack is also an ordered colection of elements like arrays, but it has a special feature that deletion and insertion of elements can be done only from one end caled the top of the stack (TOP)
- Due to this property it is also caled as last in first out type of data structure (LIFO).

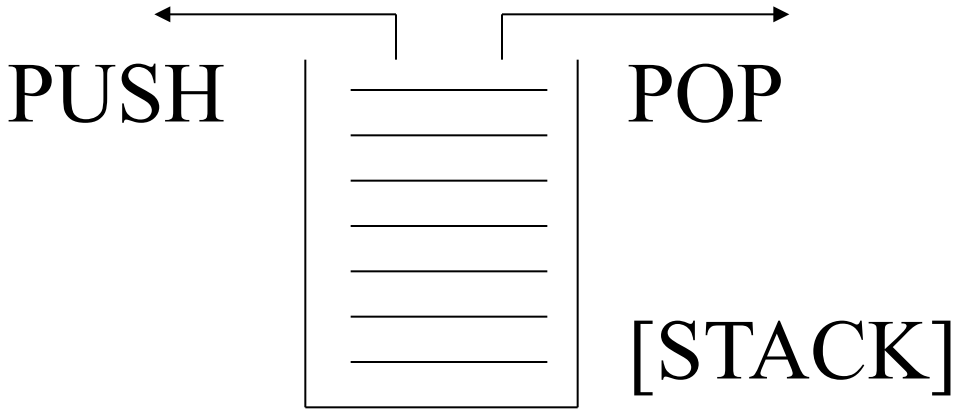


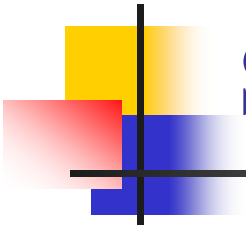
Stack

- It could be through of just like a stack of plates placed on table in a party, a guest always takes of a fresh plate from the top and the new plates are placed on to the stack at the top.
- It is a non-primitive data structure.
- When an element is inserted into a stack or removed from the stack, its base remains fixed where the top of stack changes.

Stack

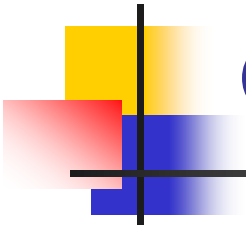
- Insertion of element into stack is caled PUSH and deletion of element from stack is caled POP.
- The below show figure how the operations take place on a stack:





Stack

- The stack can be implemented into two ways:
  - Using arrays (Static implementation)
  - Using pointer (Dynamic implementation)

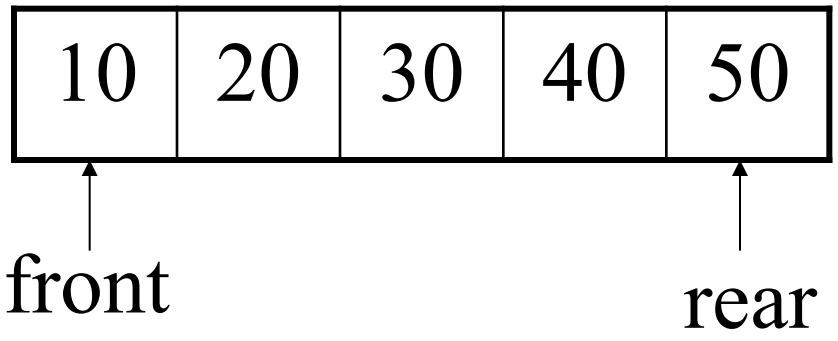


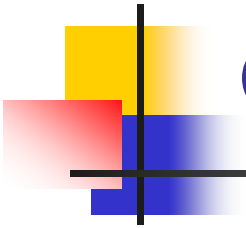
Queue

- Queue are first in first out type of data structure (i.e. FIFO)
- In a queue new elements are added to the queue from one end caled REAR end and the element are always removed from other end caled the FRONT end.
- The people standing in a railway reservation row are an example of queue.

Queue

- Each new person comes and stands at the end of the row and person geting their reservation confirmed get out of the row from the front end.
- The below show figure how the operations take place on a stack:





Queue

- The queue can be implemented into two ways:
  - Using arrays (Static implementation)
  - Using pointer (Dynamic implementation)